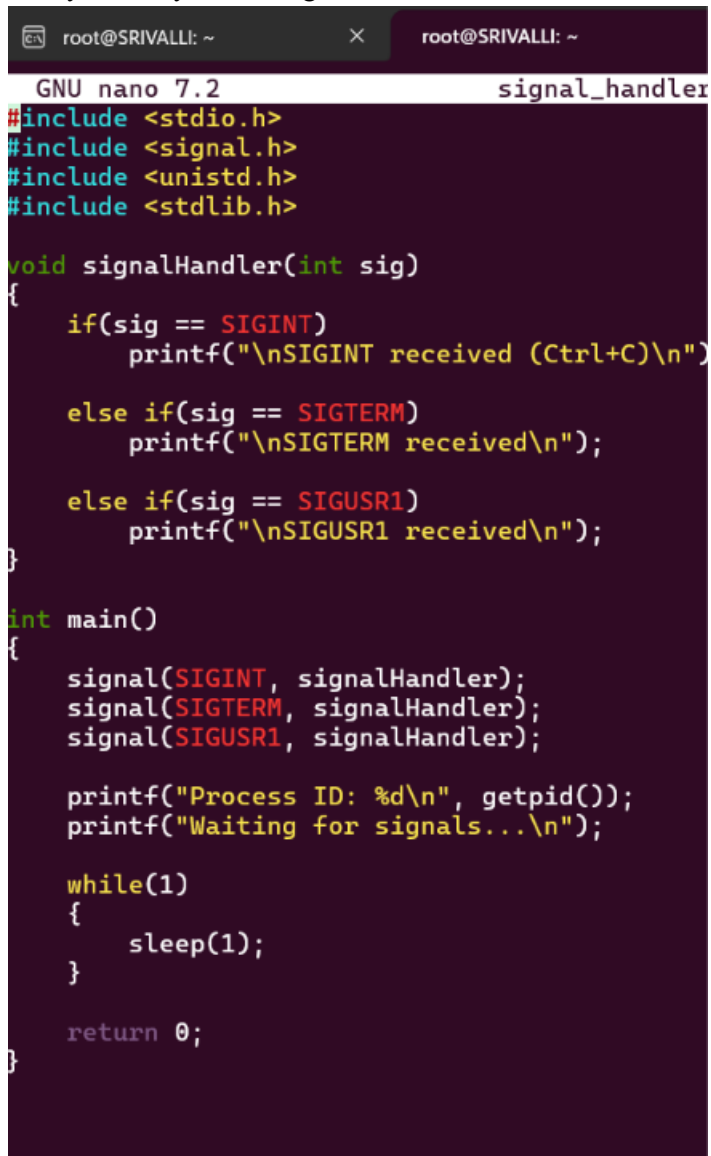


SKILL_10

1. To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test 4 Signal Events



```
GNU nano 7.2 signal_handler.c
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <stdlib.h>

void signalHandler(int sig)
{
    if(sig == SIGINT)
        printf("\nSIGINT received (Ctrl+C)\n");

    else if(sig == SIGTERM)
        printf("\nSIGTERM received\n");

    else if(sig == SIGUSR1)
        printf("\nSIGUSR1 received\n");
}

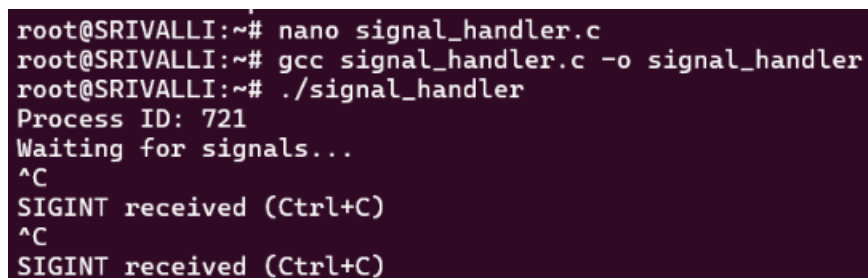
int main()
{
    signal(SIGINT, signalHandler);
    signal(SIGTERM, signalHandler);
    signal(SIGUSR1, signalHandler);

    printf("Process ID: %d\n", getpid());
    printf("Waiting for signals...\n");

    while(1)
    {
        sleep(1);
    }

    return 0;
}
```

RESULT:



```
root@SRIVALLI:~# nano signal_handler.c
root@SRIVALLI:~# gcc signal_handler.c -o signal_handler
root@SRIVALLI:~# ./signal_handler
Process ID: 721
Waiting for signals...
^C
SIGINT received (Ctrl+C)
^C
SIGINT received (Ctrl+C)
```

REPORT:

In this experiment, signal handlers were registered to handle different signals such as SIGINT, SIGTERM, SIGUSR1, and SIGUSR2. The program responded to each signal using a custom handler function. This experiment helps in understanding signal handling, interrupt management, and process stability in Linux systems.

2. To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt 4 Handhng

CODE:

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>

pid_t child_pid;

void sigint_handler(int sig)
{
    printf("\nSIGINT Captured\n");

    if(child_pid > 0)
    {
        kill(child_pid, SIGINT);
        printf("Signal Forwarded to Child Process\n");
    }
}

int main()
{
    signal(SIGINT, sigint_handler);

    child_pid = fork();

    if(child_pid == 0)
    {
        printf("Foreground Job Running...\n");

        while(1)
        {
            sleep(1);
        }
    }
    else
    {
        wait(NULL);
        printf("Job State Updated: Terminated\n");
    }
}
```

RESULT:

```
root@SRIVALLI:~# nano sigint_handling.c
root@SRIVALLI:~# gcc sigint_handling.c -o sigint_handling
root@SRIVALLI:~# ./sigint_handling
Foreground Job Running...
^C
SIGINT Captured

SIGINT Captured
Signal Forwarded to Child Process

SIGINT Captured
^C

SIGINT Captured
SIGINT Captured
Signal Forwarded to Child Process

SIGINT Captured
```

REPORT:

In this experiment, the SIGINT signal was captured using a signal handler. The signal was forwarded to the foreground process, allowing it to terminate safely while protecting the parent process. The job state was updated after termination. This experiment helps in understanding signal forwarding, interrupt handling, and job control in Linux systems.

3. To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow

CODE:

```
GNU nano 7.2 sigtstp_handling.c
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>

pid_t child_pid;

void sigtstp_handler(int sig)
{
    printf("\nSIGTSTP Captured\n");

    if(child_pid > 0)
    {
        kill(child_pid, SIGSTOP);
        printf("Child Process Suspended\n");
    }
}

int main()
{
    signal(SIGTSTP, sigtstp_handler);

    child_pid = fork();

    if(child_pid == 0)
    {
        int i = 1;

        while(1)
        {
            printf("Running... %d\n", i++);
            sleep(1);
        }
    }
    else
    {
        sleep(5);
    }
}
```

```

        printf("Running... %d\n", i++);
        sleep(1);
    }
}
else
{
    sleep(5);

    printf("Resuming Child Process...\n");
    kill(child_pid, SIGCONT);

    sleep(5);

    kill(child_pid, SIGKILL);
    wait(NULL);

    printf("Job State Updated: Completed\n");
}

return 0;
}

```

RESULT:

```

root@SRIVALLI:~# nano sigtstp_handling.c
root@SRIVALLI:~# gcc sigtstp_handling.c -o sigtstp_handling
root@SRIVALLI:~# ./sigtstp_handling
Running... 1
Running... 2
Running... 3
Running... 4
Running... 5
Resuming Child Process...
Running... 6
Running... 7
Running... 8
Running... 9
Running... 10
Job State Updated: Completed
root@SRIVALLI:~#

```

REPORT:

In this experiment, the SIGTSTP signal was captured to suspend a running process. The process state was preserved during suspension and later resumed using the SIGCONT signal. The job status was updated throughout execution. This experiment helps in understanding process suspension, job control, signal handling, and process state management in Linux systems.

4. To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior

CODE:

```
root@SRIVALLI: ~
GNU nano 7.2 process_group.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>

int main()
{
    pid_t pid;

    pid = fork();

    if(pid == 0)
    {
        setpgid(0, 0);

        printf("Child Process\n");
        printf("PID : %d\n", getpid());
        printf("PGID : %d\n", getpgrp());

        sleep(10);
    }
    else
    {
        sleep(1);

        printf("Parent Process\n");
        printf("PID : %d\n", getpid());
        printf("PGID : %d\n", getpgrp());

        wait(NULL);

        printf("Terminal Control Restored\n");
    }

    return 0;
}
```

RESULT:

```
root@SRIVALLI:~# nano process_group.c
root@SRIVALLI:~# gcc process_group.c -o process_group
root@SRIVALLI:~# ./process_group
Child Process
PID : 852
PGID : 852
Parent Process
PID : 851
PGID : 851
Terminal Control Restored
```

REPORT:

In this experiment, process groups were created using the `setpgid()` system call. The child process was assigned a separate process group ID, and process group information was displayed. The parent process coordinated execution and restored terminal control after the child process completed. This experiment helps in understanding process groups, terminal control, signal coordination, and job management in Linux systems.